Deep Manifold Structure-Preserving Spectral–Spatial Feature Extraction of Hyperspectral Image

Bing Yang, Hong Li, *Member, IEEE*, and Ziyang Guo

***Abstract*— The deep network has shown its superiority to extract discriminative features for hyperspectral image (HSI) classification. However, most existing methods only exploit label information of land classes to supervise the learning of deep features, and the learning process is totally automatic. Considering the complex spectral–spatial characteristic in real HSI, it is reasonable to utilize the manifold structure of input data to provide complementary information for precise classification. This article proposes a structure-preserving spectral–spatial network (SPSSN) to exploit the manifold structure information during the feature learning process, which can extract discriminative deep structure-preserving spectral–spatial features. Specifically, a basic spectral–spatial network (SSN), comprising a spectral module and a spatial module connected in series, is first built to learn spectral–spatial features from HSI cubes. Second, a novel structure-preserving constraint is introduced to transfer the intrinsic manifold structure of the input data into the learned deep spectral–spatial features, so as to enhance the discrimination. Furthermore, a feasible algorithm of imposing the constraint is designed to make it compatible with the deep learning framework, which results in a structure-preserving loss. Finally, the SSN is combined with the structure-preserving loss to obtain the SPSSN. Experiments on real HSI datasets verify the effectiveness and superiority of the SPSSN compared with several state-of-the-art methods in HSI classification.**

***Index Terms*— Feature extraction, hyperspectral image (HSI) classification, structure-preserving loss.**

I. Introduction

HYPERSPECTRAL image (HSI) is captured by hyperspectral sensors with the high spectral resolution, which can provide both detailed physical properties and spatial distribution of land covers [1]–[4]. A significant task in HSI analysis is HSI classification. It assigns a unique class label to each hyperspectral pixel. Different kinds of feature extraction methods have been developed to utilize plentiful spectral properties and spatial information for accurate HSI classification, such as filtering-based methods and morphology-based methods [5]–[9].

Due to the promising representation capability, deep learning-based feature extraction approaches become a trend and show excellent performance for HSI classification [10]–[13]. Earlier works focus on extract deep spectral features from 1-D spectral signatures of each pixel. In [14], hierarchical spectral features of HSI were obtained via stacked autoencoder. Hu *et al.* [15] utilized 1-D convolution neural network (CNN) to learn spectral features for classification. Mou *et al.* [16] employed recurrent neural network to analyze the hyperspectral pixels as sequential data and obtain the hierarchical features. In [17], a pixel-pair CNN is designed to explore the internal correlation of neighboring pixels. However, deep spectral features extracted only from spectral signatures are unsatisfactory because of complicated spectral variability. In fact, HSI usually contains many homogeneous regions, where the pixels within the same homogeneous region tend to belong to the same class. This high spatial correlation can be exploited to improve classification accuracy. For this, adjacent pixels are generally exploited as a 3-D HSI cube, comprising both spectral signatures and spatial correlation information. Deep learning-based spectral–spatial feature extraction methods can extract discriminative features from HSI cubes and show remarkable improvements in classification performance.

In an early study on deep learning-based spectral–spatial feature extraction, the 3-D HSI cubes are usually flattened into 1-D vectors and then fed to the neural network, such as the deep belief network and the stacked autoencoder, which destroys the inherent spatial structure. In contrast, CNN can learn spectral–spatial features naturally without vectorization and has become a popular framework for HSI classification. For example, a regularized CNN is designed to capture spectral–spatial features directly from HSI cubes [18]. Yu *et al.* [19] presented a novel CNN to deal with HSI classification in end-to-end way. Lee and Kwon [20] built a contextual CNN (CCNN), which contained a multiscale convolutional filter bank, to explore local contextual information in HSI. In [21], a deformable CNN was utilized to effectively capture the spatial structure information in HSI. More and more works tend to design deeper and more complicated networks for feature extraction of HSI. For instance, the deep feature fusion network (DFFN) was built to fuse multiple-layer features and improve classification accuracy [22]. Zhong *et al.* [23] exploited a spectral–spatial residual network (SSRN), in which spectral–spatial features are obtained by consecutive residual blocks. Sun *et al.* [24] proposed a spectral–spatial attention network to learn discriminative features from attention areas of

Manuscript received January 19, 2021; revised March 25, 2021; accepted April 7, 2021. Date of publication May 11, 2021; date of current version December 29, 2021. This work was supported by the National Natural Science Foundation of China under Grant 61877021. *(Corresponding author: Hong Li.)*

The authors are with the School of Mathematics and Statistics, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: bingyang0517@163.com; hongli@hust.edu.cn; guo_ziyang@hust.edu.cn).

Digital Object Identifier 10.1109/TGRS.2021.3072454

1558-0644 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:30:16 UTC from IEEE Xplore. Restrictions apply.

HSI cubes. Cao and Guo [25] designed a cascaded dual-scale crossover network to extract discriminant contextual features.

Nevertheless, in most methods mentioned above, the learning process of the deep features is totally automatic and supervised only by the label information of hyperspectral pixels. There is no regularization prior information embedded into the learning of deep features, which could make the learned features uncontrollable for some circumstances. A few HSI feature extraction methods have tried to impose metric learning-based constraints on the feature learning process [26]–[29]. Liu *et al.* [26] supervised Siamese CNN with a margin-based loss function to enforce the learned features with the same class to be close and those with different classes to be far away. In order to enhance the discrimination of learned deep spectral features, a center loss was combined with the cross-entropy loss to train a fully connected (FC) neural network in [27]. However, the regularization items in these methods are predefined, and they are independent of the structure of original data. As a result, these predefined regularization items may not work well when encountering complicated data structures, which is generally existed in practical HSI datasets.

Recently, many studies have shown that high-dimensional data are more likely to lie on a low-dimensional manifold embedded in the original data space [30]–[32]. Various methods have been proposed to reveal the intrinsic manifold structure of HSI and improve classification accuracy [33]–[39]. These methods attempted to extract features, which could preserve the local geometry of the manifold. For example, Zhou *et al.* [34] proposed a spatial–spectral regularized local embedding model for dimension reduction of HSI. In [35], a local neighborhood structure preserving embedding method was proposed for HSI feature extraction. However, these methods only explored the manifold structure of HSI data with shallow models. Some recent studies have successfully combined manifold learning and deep learning to extract discriminate deep features [40]–[42]. For instance, Li *et al.* [41] used a neural network to iteratively optimize the projection matrix obtained by traditional graph embedding methods, which can enhance the discrimination of embedding features. In [42], a deep locality preserving neural network (DLPNet) is proposed to integrate the extraction of deep features and the utilization of manifold structure into a unified framework. However, to simultaneously utilize label information and explore manifold structure in HSI, the output of DLPNet acted not only as the extracted deep feature but also as the predictive label vector, whose dimension must be the same as the number of classes. In contrast, this article can integrate the utilization of label information and manifold structure information in a flexible way, and the dimension of deep features can be free.

In this article, to constrain the features' learning process and utilize manifold structure information in original HSI, we introduce manifold learning into a deep learning-based HSI feature extraction framework and propose a structure-preserving spectral–spatial network (SPSSN) to extract discriminative deep structure-preserving spectral–spatial features for HSI classification. First, considering the characteristics of HSI that it contains one spectral dimension and two spatial dimensions, a spectral–spatial network (SSN) with successive spectral module and spatial module is proposed to extract deep spectral–spatial features. The spectral module consists of three successive convolutional layers along a spectral dimension, while the spatial module contains three successive convolutional layers along the spatial dimensions and a global average pooling (GAP) layer. The SSN contains an acceptable number of network parameters without using the FC layer. This can alleviate the problem of insufficient labeled training samples. Second, a manifold-based structure regularization is introduced to constrain the structure of deep features, so as to enhance the discrimination. Specifically, a novel structure-preserving constraint is imposed on the deep spectral–spatial features to preserve the intrinsic manifold structures of input data, which can be converted into a structure-preserving loss. The obtained structure-preserving loss is combined with the cross-entropy loss to supervise the learning of the deep features. Finally, the SPSSN is obtained by embedding the integrated loss into SSN. Our main contributions of this article can be highlighted as follows.

1) The SPSSN leverages local neighborhood information to regularize deep feature learning, which is dependent on the original data and can significantly enhance the representative ability of the deep spectral–spatial features.
2) The effective SSN is proposed to learn deep spectral–spatial feature representations from HSI cubes. Through successive spectral module and spatial module, the SPSSN can make full use of spectral information and spatial correlation information in HSI cubes.
3) The structure-preserving loss can be easily incorporated into the loss function of the typical deep learning pipeline and shows great universality. In addition, a simple but effective algorithm is devised to implement the structure-preserving loss, which can reduce the computational cost and guarantee feasibility.

The remainder of this article is organized as follows. Section II introduces the proposed SPSSN. Section III reports the experimental results and discussions on widely used HSI datasets, and Section IV concludes this article.

II. Deep Manifold Structure-Preserving Spectral–Spatial Feature Extraction

This section first constructs an SSN used for deep spectral–spatial feature extraction and then explains the integrated loss function, which is composed of the structure-preserving loss and cross-entropy loss. Based on the SSN and integrated loss function, an SPSSN is proposed for feature extraction of HSI.

To facilitate the following description, some notations used in this article are first defined. Let $\boldsymbol{X}$ represent an HSI with N labeled samples and B spectral bands. Let $\mathcal{L} = \{1, 2, \ldots, L\}$ denote the set of class labels for $\boldsymbol{X}$, where L is the number of classes. In this article, deep spectral–spatial feature representations are obtained from HSI cubes. Therefore, N HSI cubes centered at all labeled pixels of $\boldsymbol{X}$ are first cropped (the padding operation is used). We denote the set of all HSI cubes by $\mathcal{X} = \{\boldsymbol{X}_1, \boldsymbol{X}_2, \ldots, \boldsymbol{X}_N\}$, where $\boldsymbol{X}_i \in \mathbb{R}^{s \times s \times B}$ $(i = 1, 2, \ldots, N)$ and s represents the spatial size of HSI cubes. The class label of $\boldsymbol{X}_i$ is determined by

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:30:16 UTC from IEEE Xplore. Restrictions apply.

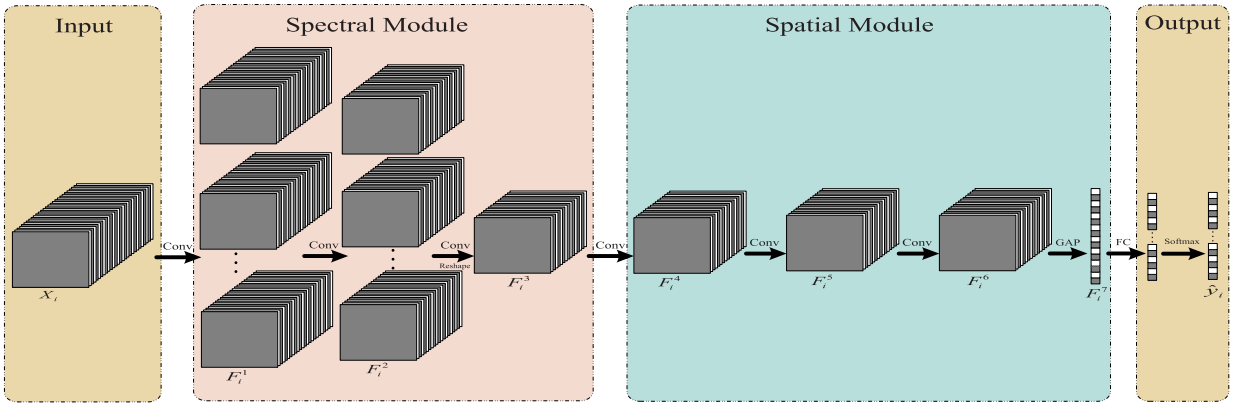


Fig. 1. Architecture of the proposed SSN. The SSN comprises successive spectral module and spatial module. The spectral module is composed of three convolutional (Conv) layers, and the spatial module is composed of three convolutional layers and a GAP layer. The spectral–spatial features extracted by the SSN are finally fed to an FC layer and a softmax function for classification.

that of its central pixel, and the label set of $\mathcal{X}$ is denoted by $\mathcal{Y} = \{y_1, y_2, \ldots, y_N\}$, where $y_i \in \mathcal{L}$ $(i = 1, 2, \ldots, N)$.

A. Spectral–Spatial Network

Considering the characteristics of HSI that it contains one spectral dimension and two spatial dimensions, the SSN with successive spectral and spatial modules is designed to obtain deep spectral–spatial features from HSI cubes. Fig. 1 shows the network architecture of the proposed SSN, and the details are introduced as follows.

1) Spectral Module: Hundreds of spectral bands of HSI can provide abundant physical properties of land covers. To explore detailed spectral information, the spectral module is first built to obtain the deep spectral features of each spectral signature in HSI cubes alone spectral dimension.

The spectral module is composed of three convolutional layers with C kernels in each layer. The spectral module focuses on extracting spectral features of each hyperspectral pixel, so the convolutional kernels in the spectral module are all set with the spatial size of 1×1. In addition, to alleviate the computational cost and reduce the dimensionality of extracted spectral features, the padding operation is not used in the spectral module, and the convolutional stride along a spectral dimension is set to 2. The details are illustrated as follows.

The HSI cube $\boldsymbol{X}_i$ is first fed to the spectral module. In the first convolutional layer of spectral module (abbreviated as the first spectral convolutional layer), $\boldsymbol{X}_i \in \mathbb{R}^{s\times s\times B}$ is convoluted with kernel $\boldsymbol{W}^{1,t} \in \mathbb{R}^{1\times 1\times 7}$ to generate $\boldsymbol{F}_i^{1,t} \in \mathbb{R}^{s\times s\times c_1}$, where $c_1 = (B-7)/2+1$ and $t = 1, 2, \ldots, C$. Then, $\boldsymbol{F}_i^{1,t}$ are stacked along the last dimension to generate $\boldsymbol{F}_i^1$

$$\boldsymbol{F}_i^{1,t} = \phi\left(\boldsymbol{W}^{1,t} * \boldsymbol{X}_i + b^{1,t}\right), \quad t = 1, 2, \ldots, C \tag{1}$$

$$\boldsymbol{F}_i^1 = \left\langle \boldsymbol{F}_i^{1,1}, \boldsymbol{F}_i^{1,2}, \ldots, \boldsymbol{F}_i^{1,C} \right\rangle \tag{2}$$

where $*$ represents the convolution operation, $b^{1,t} \in \mathbb{R}^1$ is the bias, $\langle\cdot\rangle$ represents the stacking operation, and ϕ is the rectified linear unit (ReLU) activation function. For simplicity, the formula is denoted as

$$\boldsymbol{F}_i^1 = \xi\left(\boldsymbol{W}^1 * \boldsymbol{X}_i + \boldsymbol{b}^1\right) \tag{3}$$

where $\boldsymbol{W}^1$ and $\boldsymbol{b}^1$ represent the weight and bias parameters of the first spectral convolutional layer, respectively. The obtained $\boldsymbol{F}_i^1$ is of size $s \times s \times c_1 \times C$.

In the second spectral convolutional layer with C convolutional kernels of size $1 \times 1 \times 7$, $\boldsymbol{F}_i^2$ is obtained by

$$\boldsymbol{F}_i^2 = \xi\left(\boldsymbol{W}^2 * \boldsymbol{F}_i^1 + \boldsymbol{b}^2\right) \tag{4}$$

where $\boldsymbol{W}^2$ and $\boldsymbol{b}^2$ are weight and bias of the second spectral convolutional layer, respectively. $\boldsymbol{F}_i^2$ is of size $s \times s \times c_2 \times C$, where $c_2 = (c_1 - 7)/2 + 1$.

In the third spectral convolutional layer with C convolutional kernels of size $1 \times 1 \times c_2$, $\hat{\boldsymbol{F}}_i^3$ is obtained by

$$\hat{\boldsymbol{F}}_i^3 = \xi\left(\boldsymbol{W}^3 * \boldsymbol{F}_i^2 + \boldsymbol{b}^3\right) \tag{5}$$

where $\boldsymbol{W}^3$ and $\boldsymbol{b}^3$ are weight and bias, respectively, and $\hat{\boldsymbol{F}}_i^3$ is of size $s \times s \times 1 \times C$. Then, $\hat{\boldsymbol{F}}_i^3$ is reshaped to obtain the output of the spectral module $\boldsymbol{F}_i^3 \in \mathbb{R}^{s\times s\times C}$.

2) Spatial Module: The spectral module can capture spectral features of the HSI cube. However, as mentioned above, the spectral variability is complicated, and the spectral features extracted only from 1-D spectral signatures are unsatisfactory. For this, we propose the spatial module to fuse spectral features and further obtain spectral–spatial features.

The spatial module is composed of three convolutional layers and a GAP layer. Three convolutional layers in the spatial module all have C convolutional kernels of size 3×3. The padding operation is applied in the spatial module. The detailed architecture is described as follows.

In the lth convolutional layer of the spatial module, the input $\boldsymbol{F}_i^{l-1}$ is first convoluted with kernel $\boldsymbol{W}^{l,t} \in \mathbb{R}^{3\times 3}$ to obtain $\boldsymbol{F}_i^{l,t} \in \mathbb{R}^{s\times s}$, where $l = 4, 5, 6$ and $t = 1, 2, \ldots, C$. Then, $\boldsymbol{F}_i^{l,t}(t = 1, 2, \ldots, C)$ are concatenated to generate $\boldsymbol{F}_i^l \in \mathbb{R}^{s\times s\times C}$

$$\boldsymbol{F}_i^{l,t} = \phi\left(\boldsymbol{W}^{l,t} * \boldsymbol{F}_i^{l-1} + b^{l,t}\right), \quad t = 1, 2, \ldots, C \tag{6}$$

$$\boldsymbol{F}_i^l = \left[\boldsymbol{F}_i^{l,1}, \boldsymbol{F}_i^{l,2}, \ldots, \boldsymbol{F}_i^{l,C}\right] \tag{7}$$

where $b^{l,t} \in \mathbb{R}^1$ is the bias and $[\cdot]$ represents the concatenating operation along channel.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:30:16 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PARAMETERS AND FLOPS OF THE SPSSN

	SpeConv1	SpeConv2	SpeConv3	SpaConv1	SpaConv2	SpaConv3	Output	sum
Parameters	$8C$	$7C^2+C$	c_2C^2+C	$9C^2+C$	$9C^2+C$	$9C^2+C$	$L(C+1)$	$(34+c_2)C^2+(13+L)C+L$
FLOPs	$14Cs^2c_1$	$14C^2s^2c_2$	$2c_2C^2s^2$	$18C^2s^2$	$18C^2s^2$	$18C^2s^2$	$2LC$	$(54+16c_2)C^2s^2+(14s^2c_1+2L)C$

[1] SpeConv1, SpeConv2 and SpeConv3 represent the first, second and third spectral convolutional layers, respectively. SpaConv1, SpaConv2 and SpaConv3 represent the first, second and third spatial convolutional layers, respectively.

Finally, $\boldsymbol{F}_i^6$ is fed to a GAP layer and a flattening layer to obtain $\boldsymbol{F}_i^7 \in \mathbb{R}^C$

$$\boldsymbol{F}_i^7 = \text{fla}\big(\text{gap}\big(\boldsymbol{F}_i^6\big)\big) \tag{8}$$

where fla$(\cdot)$ represents the flattening operation and gap$(\cdot)$ represents the global-average-pooling operation.

$\boldsymbol{F}_i^7$ is actually the deep spectral–spatial features extracted by the SSN, and the overall feature extraction process can be summarized as

$$\boldsymbol{X}_i \mapsto \boldsymbol{F}_i^1 \mapsto \boldsymbol{F}_i^2 \mapsto \boldsymbol{F}_i^3 \mapsto \boldsymbol{F}_i^4 \mapsto \boldsymbol{F}_i^5 \mapsto \boldsymbol{F}_i^6 \mapsto \boldsymbol{F}_i^7. \tag{9}$$

Note that the SSN contains an acceptable number of network parameters without using the FC layer. This can alleviate the problem of insufficient labeled training samples. Furthermore, batch normalization is applied to every layer in the SSN.

Next, the computational complexity of the proposed network architecture is analyzed in terms of the number of network parameters and the number of floating-point operations (FLOPs). The results are summarized in Table I. Note that the complexity is reported with that of batch normalization omitted.

Remark 1: The number of the FLOPs of the SPSSN is $(54+16c_2)s^2C^2+(14s^2c_1+2L)C$, and the number of the parameters of the SPSSN is $(34+c_2)C^2+(13+L)C+L$, where L is the number of classes, s is the spatial size of HSI cube, C is the number of convolutional kernels, $c_1=(B-7)/2+1$, $c_2=(c_1-7)/2+1$, and B is the number of spectral bands.

Remark 1 gives the computational complexity of SPSSN, which is related to the size of the input HSI cube and the number of convolutional kernels C. For a specific HSI dataset, B, L, and s are fixed, and then, the overall complexity is dominated by C. Though a large C is helpful to obtain more discriminative feature representations, this also means more computational burden. In order to control the computation cost and simultaneously extract discriminative deep features, a moderate value of C should be chosen in practice.

B. Joint Supervision of Cross-Entropy Loss and Structure-Preserving Loss

After extracting the spectral–spatial feature $\boldsymbol{F}_i^7$ from the HSI cube $\boldsymbol{X}_i$, the softmax classifier is employed for HSI classification. The softmax classifier is an FC layer followed by a softmax activation function, which is formulated as

$$\hat{\boldsymbol{y}}_i = \varphi\big(\boldsymbol{W}^7\boldsymbol{F}_i^7+\boldsymbol{b}^7\big) \tag{10}$$

where $\boldsymbol{W}^7 \in \mathbb{R}^{L\times C}$ and $\boldsymbol{b}^7 \in \mathbb{R}^L$ are the weight and bias, respectively, and φ represents the softmax function. The class label of the HSI cube $\boldsymbol{X}_i$ can be predicted by

$$\text{label}(\boldsymbol{X}_i) = \underset{j\in\{1,2,\ldots,L\}}{\text{argmax}}\ \hat{y}_{ij} \tag{11}$$

where $\hat{y}_{ij}$ is the jth element of $\hat{\boldsymbol{y}}_i$.

1) Cross-Entropy Loss: Let D_{N_b} denote the set of the training samples in the current minibatch, and N_b is the size of the minibatch. Let $\mathcal{I}_{N_b} = \{m|\boldsymbol{X}_m \in D_{N_b}\}$ denote the set of indexes of training samples in current minibatch. The conventional objective function of training the aforementioned network is cross-entropy loss, which aims to minimize the classification error and is defined as

$$\mathcal{J}_{ce}(\boldsymbol{W},\boldsymbol{b}) = \frac{1}{N_b}\sum_{i\in\mathcal{I}_{N_b}}\left\{-\sum_{j=1}^{L}\mathbb{I}_{\{j\}}(y_i)\log\hat{y}_{ij}\right\} \tag{12}$$

where $\boldsymbol{W}=\{\boldsymbol{W}^1,\boldsymbol{W}^2,\ldots,\boldsymbol{W}^7\}$, $\boldsymbol{b}=\{\boldsymbol{b}^1,\boldsymbol{b}^2,\ldots,\boldsymbol{b}^7\}$, and $\mathbb{I}_{\mathcal{A}}(x)$ is the indicator function of set $\mathcal{A}$

$$\mathbb{I}_{\mathcal{A}}(x) = \begin{cases}1, & \forall x\in\mathcal{A}\\ 0, & \text{otherwise.}\end{cases} \tag{13}$$

As previously mentioned, the cross-entropy loss only exploits the label information to supervise the deep feature learning, and the learned features are uncontrollable because no prior is embedded into the learning process. Nevertheless, the real HSI is very complicated, and it is insufficient to learn discriminative features that supervising the network only by the label information.

2) Structure-Preserving Loss: Apart from the label information, the manifold structure of the original HSI can be used to provide complemental information to constrain the learning process and enhance the discrimination of learned deep features.

In this article, we constrain the learning process of deep features by preserving the manifold structure of the HSI as much as possible. Specifically, motivated by the Laplacian eigenmaps (LEs) [43], we hope to preserve local neighborhood information of original HSI in the learned deep features. This means that, if $\boldsymbol{X}_j$ is close to $\boldsymbol{X}_i$, then the corresponding deep feature $\boldsymbol{F}_j^m$ is also close to $\boldsymbol{F}_i^m$, where $m \in \{1,2,\ldots,7\}$. Without loss of generality, in this work, we only impose this structure-preserving constraint on the deep features of the top layer. It is also reasonable to impose the structure-preserving constraint on the deep features of the top layer because error propagation is from the top layer to the input layer during the training process.

Denote $k-\text{NN}(\boldsymbol{X}_i)$ as the set of k-nearest neighbors of $\boldsymbol{X}_i$. Denote $\mathcal{N}(\boldsymbol{X}_i)=\{m|\boldsymbol{X}_m \in k-\text{NN}(\boldsymbol{X}_i)\}$ as the set of indexes of the k-nearest neighbors of $\boldsymbol{X}_i$. Our goal can be formulated by minimizing the following objective function, termed structure-preserving loss:

$$\mathcal{J}'_{sp}(\boldsymbol{W},\boldsymbol{b}) = \frac{1}{kN}\sum_{i=1}^{N}\sum_{j\in\mathcal{N}(\boldsymbol{X}_i)}\|\boldsymbol{F}_i-\boldsymbol{F}_j\|^2A_{ij} \tag{14}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:30:16 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Calculating the Structure-Preserving Loss, Which Is Denoted as $\mathcal{J}_{sp} = spl(D_{N_b}, F_{N_b}, D_B, F_B, k)$

Input: D_{N_b},F_{N_b},D_B, F_B, k.
Output: $\mathcal{J}_{sp}$.
1 $t \leftarrow \max_{\boldsymbol{X}_i \in D_{N_b}, \boldsymbol{X}_j \in D_B} \|\boldsymbol{X}_i - \boldsymbol{X}_j\|^2$;
2 **for** $i \in \mathcal{I}_{N_b}$ **do**
3 **for** $j \in \{m | \boldsymbol{X}_m \in D_B^{l_i}\}$ **do**
4 $A_{i,j} \leftarrow e^{-\frac{\|\boldsymbol{X}_i - \boldsymbol{X}_j\|^2}{t}}$;
5 **end**
6 $A_i \leftarrow \sum_j A_{i,j}$;
7 **for** $j \in \{m | \boldsymbol{X}_m \in D_B^{l_i}\}$ **do**
8 $A_{i,j} \leftarrow A_{i,j}/A_i$;
9 **end**
10 **end**
11 $\mathcal{J}_{sp} \leftarrow \frac{1}{kN_b} \sum_{i \in \mathcal{I}_{N_b}} \sum_{\boldsymbol{F}_j \in F_B^{l_i}} \|\boldsymbol{F}_i - \boldsymbol{F}_j\|^2 \ A_{ij}$;
12 **return** $\mathcal{J}_{sp}$.

where $\boldsymbol{F}_i = \boldsymbol{F}_i^7$, and $A_{i,j}$ is defined as

$$A_{i,j} = \begin{cases} K(\boldsymbol{X}_i, \boldsymbol{X}_j), & \text{if } \boldsymbol{X}_j \in k - \mathrm{NN}(\boldsymbol{X}_i) \\ 0, & \text{otherwise} \end{cases} \tag{15}$$

where

$$K(\boldsymbol{X}_i, \boldsymbol{X}_j) = e^{-\frac{\|\boldsymbol{X}_i - \boldsymbol{X}_j\|^2}{t}} \tag{16}$$

which is the heat kernel, and t is the kernel parameter. The heat kernel can describe the similarity of samples in original input space.

Ideally, the local neighborhood information of all training data should be taken into account, which is inefficient even impractical. To address this problem, we make two necessary modifications. First, instead of preserving neighborhood information with respect to the entire training set, we make the constraint only on the minibatch. Let F_{N_b} denote the set of the corresponding deep features of D_{N_b}. Let $D_{N_b}^l = \{\boldsymbol{X}_i | \boldsymbol{X}_i \in D_{N_b} \wedge l_i = l\}$ and $F_{N_b}^l = \{\boldsymbol{F}_i | \boldsymbol{F}_i \in F_{N_b} \wedge l_i = l\}$ denote the sets of the training samples and corresponding features for class l in current minibatch, respectively, where l_i denotes the label of $\boldsymbol{X}_i$ and $l \in \{1, 2, \ldots, L\}$. Equation (14) is then converted to

$$\mathcal{J}''_{sp}(\boldsymbol{W}, \boldsymbol{b}) = \frac{1}{kN_b} \sum_{i \in \mathcal{I}_{N_b}} \sum_{j \in \mathcal{N}(\boldsymbol{X}_i)} \|\boldsymbol{F}_i - \boldsymbol{F}_j\|^2 A_{ij} \tag{17}$$

where $A_{i,j}$ is the distance between $\boldsymbol{X}_i \in D_{N_b}$ and $\boldsymbol{X}_j \in k - \mathrm{NN}(\boldsymbol{X}_i)$.

Our second modification is to avoid finding the neighbors of training data in minibatch during each iteration. It should be noted that the pixels that belong to the same class are more likely to have similar spectral curves, which means that they are more likely to be the neighbors of each other. Therefore, a data buffer set and a corresponding feature buffer set are constructed to ease the computation of structure-preserving loss (the construction details are illustrated in the following). Let D_B^l and F_B^l denote the sets of the data buffer and the corresponding feature buffer for class l, respectively,

Algorithm 2 Updating the Data Buffer and the Corresponding Feature Buffer, Which Is Denoted as $\{\hat{D}_B, \hat{F}_B\} = update(D_B, F_B, D_{N_b}, F_{N_b}, k)$

Input: D_B, F_B, D_{N_b}, F_{N_b}, k.
Output: $\hat{D}_B$, $\hat{F}_B$.
1 **for** $l \in \{1, 2, \ldots, L\}$ **do**
2 **if** $|D_{N_b}^l| \geq k$ **then**
3 $\breve{D}_B^l \sim s(D_{N_b}^l, k)$, $\breve{F}_B^l \sim s(F_{N_b}^l, k)$;
4 **else**
5 $\breve{D}_B^l \leftarrow s(D_B^l, k - |D_{N_b}^l|)$, $\breve{F}_B^l \leftarrow s(F_B^l, k - |D_{N_b}^l|)$;
6 $\breve{D}_B^l \leftarrow \breve{D}_B^l \cup D_{N_b}^l$, $\breve{F}_B^l \leftarrow \breve{F}_B^l \cup F_{N_b}^l$;
7 **end**
8 **end**
9 $\breve{D}_B \leftarrow \bigcup_{l=1}^L \breve{D}_B^l$, $\breve{F}_B \leftarrow \bigcup_{l=1}^L \breve{F}_B^l$;
10 $\mathcal{J}_{sp} \leftarrow spl(D_{N_b}, F_{N_b}, D_B, F_B, k)$;
11 $\breve{\mathcal{J}}_{sp} \leftarrow spl(D_{N_b}, F_{N_b}, \breve{D}_B, \breve{F}_B, k)$;
12 **if** $\breve{\mathcal{J}}_{sp} \geq \mathcal{J}_{sp}$ **then**
13 $\{\hat{D}_B, \hat{F}_B\} \leftarrow \{D_B, F_B\}$;
14 **else**
15 $\{\hat{D}_B, \hat{F}_B\} \leftarrow \{\breve{D}_B, \breve{F}_B\}$;
16 **end**
17 **return** $\hat{D}_B$, $\hat{F}_B$.

where $l \in \{1, 2, \ldots, L\}$. We always keep k samples in the data (feature) buffer for each class. Let $D_B \equiv \bigcup_{l=1}^L D_B^l$ and $F_B \equiv \bigcup_{l=1}^L F_B^l$. Then, (17) is converted to

$$\mathcal{J}_{sp}(\boldsymbol{W}, \boldsymbol{b}) = \frac{1}{kN_b} \sum_{i \in \mathcal{I}_{N_b}} \sum_{\boldsymbol{F}_j \in F_B^{l_i}} \|\boldsymbol{F}_i - \boldsymbol{F}_j\|^2 A_{ij} \tag{18}$$

where l_i denotes the label of $\boldsymbol{X}_i$, and $A_{i,j}$ is the distance between $\boldsymbol{X}_i \in D_{N_b}$ and $\boldsymbol{X}_j \in D_B^{l_i}$. Algorithm 1 shows the details of calculating the structure-preserving loss.

Before calculating the structure-preserving loss in each training iteration, the data buffer set and the corresponding feature buffer set should be updated. The complete procedures are shown in Algorithm 2. The input of this algorithm includes the old data buffer set D_B, the old feature buffer set F_B, the set of training samples in current minibatch D_{N_b}, the set of corresponding deep features F_{N_b}, and the size of the buffer set for each class k. The output of this algorithm is the updated data buffer set $\hat{D}_B$ and the updated feature buffer set $\hat{F}_B$. For the first iteration, we set $D_B = D_{N_b}$ and $F_B = F_{N_b}$.

Let $|\mathcal{A}|$ denote the number of elements in the finite set $\mathcal{A}$. Let $s(\mathcal{A}, n)$ denote the operation that randomly selects n elements from the set $\mathcal{A}$. Then, the detailed steps of Algorithm 2 can be described as follows.

Step 1 (Constructing the Candidate Data Buffer $\breve{D}_B$ and the Candidate Feature Buffer $\breve{F}_B$, Lines 1–9): First, we construct the candidate data buffer of the lth class, denoted by $\breve{D}_B^l$, and the candidate feature buffer of the lth class, denoted by $\breve{F}_B^l$, where $l = 1, 2, \ldots, L$ (lines 1–8). The criteria of constructing $\breve{D}_B^l$ and $\breve{F}_B^l$ are defined as follows: if the number of training samples of the lth class, i.e., $|D_{N_b}^l|$, is larger than or equal to k, we randomly choose k training samples and corresponding deep features from $D_{N_b}^l$ and $F_{N_b}^l$ (lines 2 and 3); otherwise,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:30:16 UTC from IEEE Xplore. Restrictions apply.

we first randomly choose $k - |D_{N_b}^l|$ training samples and corresponding deep features from D_B^l and F_B^l, respectively, and then combine the obtained samples and features with $D_{N_b}^l$ and $F_{N_b}^l$ (lines 4–6). Finally, the candidate data buffer $\breve{D}_B$ and the candidate feature buffer $\breve{F}_B$ are obtained as $\breve{D}_B = \bigcup_{l=1}^{L} \breve{D}_B^l$ and $\breve{F}_B = \bigcup_{l=1}^{L} \breve{F}_B^l$ (line 9).

Step 2 (Calculating the Condition Used for Updating Data Buffer and Feature Buffer, Lines 10 and 11): Line 10 is used to calculate the structure-preserving loss based on the old data buffer D_B and the old feature buffer F_B, denoted by $\mathcal{J}_{sp}$, and line 11 is used to calculate the structure-preserving loss based on the candidate data buffer $\breve{D}_B$ and the candidate feature buffer $\breve{F}_B$, denoted by $\breve{\mathcal{J}}_{sp}$.

Step 3 (Updating Data Buffer and Feature Buffer, lines 12–16): If $\breve{\mathcal{J}}_{sp}$ is larger than or equal to $\mathcal{J}_{sp}$ (line 12), then we update the buffers as $\{\hat{D}_B, \hat{F}_B\} \leftarrow \{D_B, F_B\}$ (line 13); otherwise, we update the buffers as $\{\hat{D}_B, \hat{F}_B\} \leftarrow \{\breve{D}_B, \breve{F}_B\}$ (line 15).

Step 4 (Returning Data Buffer and Feature Buffer, line 17): Return the updated buffers $\hat{D}_B$ and $\hat{F}_B$.

3) Integrated Loss: The structure-preserving loss is then combined with the cross-entropy loss to obtain the final objective function

$$\begin{aligned}&\mathcal{J}(\boldsymbol{W}, \boldsymbol{b})\\&= \mathcal{J}_{ce}(\boldsymbol{W}, \boldsymbol{b}) + \lambda \mathcal{J}_{sp}(\boldsymbol{W}, \boldsymbol{b})\\&= \frac{1}{N_b} \sum_{i \in \mathcal{I}_{N_b}} \left\{ -\sum_{j=1}^{L} \mathbb{I}_{\{j\}}(y_i) \log \hat{y}_{ij} + \frac{\lambda}{k} \sum_{\boldsymbol{F}_j \in F_B^{l_i}} \|\boldsymbol{F}_i - \boldsymbol{F}_j\|^2 A_{ij} \right\}\end{aligned} \tag{19}$$

where the scalar λ is adopted to balance the cross-entropy loss and the structure-preserving loss.

Algorithm 3 Training Algorithm of the SPSSN

Input: Training set D_N, the batch size N_b, the hyperparameter λ, the learning rate η, the size of buffer set for each class k, the number of maximal iterations τ_{max}, the index of iteration m.
Output: $\boldsymbol{W}$ and $\boldsymbol{b}$.

1 Initialize $m = 1$, $\boldsymbol{W}$ and $\boldsymbol{b}$ with random values;
2 **repeat**
3 $D_{N_b} \sim s(D_N, N_b)$;
4 $F_{N_b} \leftarrow \varnothing$, where $\varnothing$ denotes empty set;
5 **for** $\boldsymbol{X}_i \in D_{N_b}$ **do**
6 compute $\boldsymbol{F}_i^7, \hat{\boldsymbol{y}}_i$;
7 $\boldsymbol{F}_i \leftarrow \boldsymbol{F}_i^7$;
8 $F_{N_b} \leftarrow F_{N_b} \cup \boldsymbol{F}_i$;
9 **end**
10 **if** $m = 1$ **then**
11 **for** $l \in \{1, 2, \ldots, L\}$ **do**
12 $D_B^l \leftarrow \{\boldsymbol{X}_i | \boldsymbol{X}_i \in D_{N_b} \wedge \text{label}(\boldsymbol{X}_i) = l\}$;
13 $F_B^l \leftarrow \{\boldsymbol{F}_i | \boldsymbol{F}_i \in F_{N_b} \wedge \text{label}(\boldsymbol{F}_i) = l\}$;
14 **end**
15 $D_B \leftarrow \bigcup_{l=1}^{L} D_B^l$, $F_B \leftarrow \bigcup_{l=1}^{L} F_B^l$
16 **end**
17 $\{D_B, F_B\} \leftarrow update(D_B, F_B, D_{N_b}, F_{N_b}, k)$;
18 $\mathcal{J}_{ce}(\boldsymbol{W}, \boldsymbol{b}) \leftarrow \frac{1}{N_b} \sum_{i \in \mathcal{I}_{N_b}} \{-\sum_{j=1}^{L} \mathbb{I}_{\{j\}}(y_i) \log \hat{y}_{ij}\}$;
19 $\mathcal{J}_{sp}(\boldsymbol{W}, \boldsymbol{b}) \leftarrow spl(D_{N_b}, F_{N_b}, D_B, F_B, k)$;
20 $\mathcal{J}(\boldsymbol{W}, \boldsymbol{b}) \leftarrow \mathcal{J}_{ce}(\boldsymbol{W}, \boldsymbol{b}) + \lambda \mathcal{J}_{sp}(\boldsymbol{W}, \boldsymbol{b})$;
21 $\nabla_{\boldsymbol{W}} \leftarrow \frac{\partial \mathcal{J}(\boldsymbol{W}, \boldsymbol{b})}{\partial \boldsymbol{W}}$, $\nabla_{\boldsymbol{b}} \leftarrow \frac{\partial \mathcal{J}(\boldsymbol{W}, \boldsymbol{b})}{\partial \boldsymbol{b}}$;
22 $\boldsymbol{W} \leftarrow \boldsymbol{W} - \eta \nabla_{\boldsymbol{W}}$, $\boldsymbol{b} \leftarrow \boldsymbol{b} - \eta \nabla_{\boldsymbol{b}}$;
23 $m \leftarrow m + 1$;
24 **until** $m > \tau_{max}$;
25 **return** $\boldsymbol{W}$, $\boldsymbol{b}$.

C. Deep Manifold Structure-Preserving Spectral–Spatial Feature Extraction Based on SPSSN

By combining the SSN and the integrated loss, we can obtain the SPSSN. Benefitting from the structure-preserving loss, the SPSSN can extract deep structured spectral–spatial features that have significant separability and can improve the performance of subsequent HSI classification tasks. The procedures for training SPSSN are summarized in Algorithm 3.

The whole HSI feature extraction and classification framework based on the SPSSN is depicted in Fig. 2. Specifically, the proposed framework contains a training stage and a testing stage, which are distinguished by different dashed rectangles. During the training stage, the data buffer set and the feature buffer set are first built based on the training data and the SPSSN. Then, the cross-entropy loss and the structure-preserving loss are calculated and integrated to optimize the network parameters of the SPSSN. During the testing stage, the trained SPSSN can be used to obtain both the discriminant deep feature and the predicted label of the testing data.

Remark 2: Assume that the HSI $\boldsymbol{X}$ is a compact smooth n-dimensional manifold. Then, $\boldsymbol{X}$ can be embedded in $\mathbb{R}^{2n+1}$.

Let the HSI $\boldsymbol{X}$ be a compact smooth n-dimensional manifold. Then, according to the Whitney embedding theorem [44], we can claim that $\boldsymbol{X}$ can be embedded in $\mathbb{R}^{2n+1}$. In this article, we focus on obtaining the low-dimension manifold representations of original HSI, whose pixels have B spectral bands. In other words, the HSI is now embedded in $\mathbb{R}^B$. According to Remark 2, it is reasonable to assume that the HSI is a $(B-1)/2$-dimensional manifold. Such a result is very useful to guide the design of the network architecture of the SPSSN. Specifically, in this work, $\boldsymbol{F}^7 \in \mathbb{R}^C$ is the deep spectral–spatial feature extracted by the SPSSN, which means that the dimension of the manifold feature space is C. Therefore, C can be approximately chosen around $(B-1)/2$. The experimental evidence of this phenomenon is to be accessed in Section III-B.

III. Experimental Results and Discussion

A. Datasets' Description and Experimental Settings

1) Datasets: The effectiveness of the SPSSN is verified on four representative HSI datasets, i.e., Salinas, Indian Pines, Houston2013, and Pavia University.

The Salinas dataset was collected by AVIRIS sensor over Salinas Valley, California, and this scene consists of 512 × 217 pixels and 204 spectral bands with a spatial resolution of 3.7 m. There are 16 classes in this scene.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:30:16 UTC from IEEE Xplore. Restrictions apply.

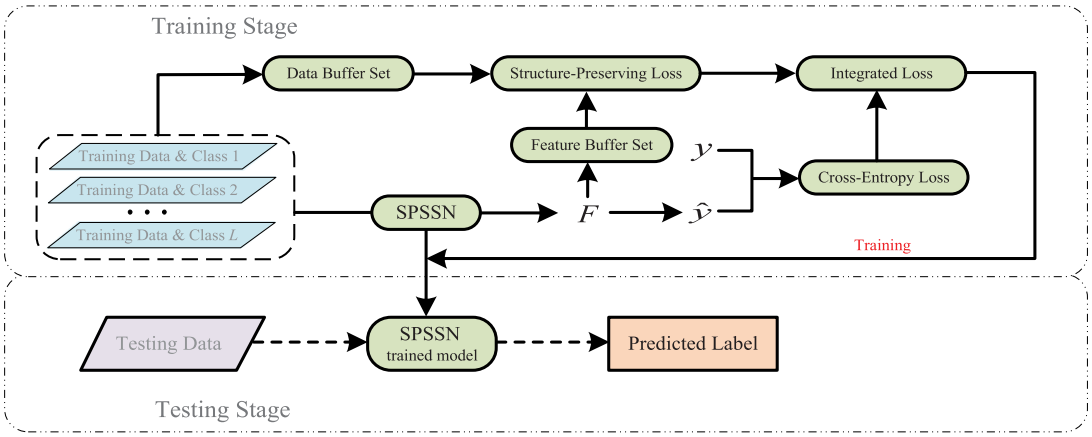


Fig. 2. SPSSN-based HSI feature extraction and classification framework.

The Indian Pines dataset was gathered over the Indian Pines test site in Indiana by the AVIRIS sensor. This dataset is composed of 145 × 145 pixels with 200 spectral bands, and the spatial resolution is 20 m. The number of considered classes in this scene is 16.

The Houston2013 dataset was acquired over the campus of University of Houston in USA, and its size is 349 × 1905 with 144 spectral channels. The spatial resolution is 2.5 m per pixel. This dataset considers 15 challenging land-cover classes, and the setting of training and test sets is fixed.

The Pavia University dataset was recorded by ROSIS sensor over Pavia, Italy. This dataset comprises 610 × 340 pixels and 103 spectral bands with a spatial resolution of 1.3 m. Only nine classes of interest are considered for this scene.

2) Experimental Settings: In the SPSSN, the structure-preserving loss coefficient λ is set to 0.5 for four datasets. The size of buffer set for each class, i.e., k, is set to 10, 10, 10, and 5 for the Salinas, Indian Pines, Houston2013, and Pavia University datasets, respectively. The number of convolutional kernels C is set to 100, 100, 75, and 50 for the Salinas, Indian Pines, Houston2013, and Pavia University datasets, respectively. These parameters are analyzed in Section III-B. The spatial size of the HSI cube is set to 7 × 7 for four datasets. If not specified, the training percentages on Salinas, Indian Pines, and Pavia University datasets are set to 1%, 10%, and 1%, respectively. The effects of different spatial sizes and training percentages are illustrated in Section III-E. In addition, the same training settings are adopted on four datasets. The SPSSN is trained by the Adam optimizer [45] for 30k iterations. The learning rate η is set to 0.0007. The size of minibatch, i.e., N_b, is set to 100.

Four common metrics are used to quantify the classification performance, which are the overall accuracy (OA), average accuracy (AA), kappa coefficient (κ), and individual class accuracy. These metrics are reported by averaging the results of ten independent experiments. All experiments are implemented on a desktop equipped with Intel Core i7-6700 CPU and GeForce GTX 1060.

B. Parameter Analysis

In this section, the roles of the crucial parameters, i.e., the structure-preserving loss coefficient λ, the size of the buffer set for each class k, and the number of convolutional kernels C, are analyzed.

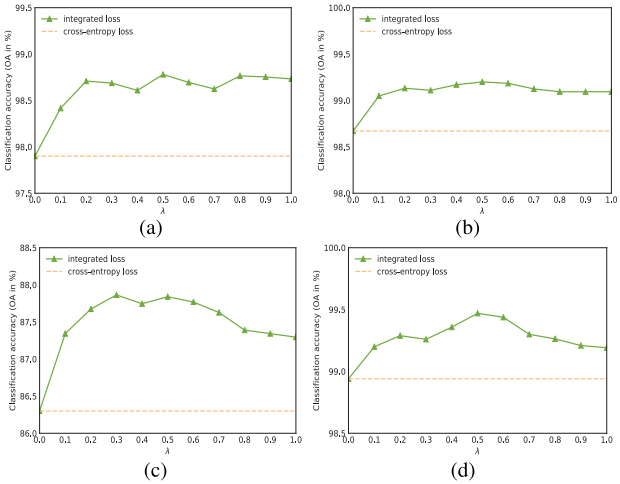


Fig. 3. Effect of different λ's. (a) Salinas. (b) Indian Pines. (c) Houston2013. (d) Pavia University.

First, we study the effect of different λ on the OA. The parameter λ controls the importance of structure-preserving loss on deep features' learning. The optimal value of λ is selected within the given set $\{0.0, 0.1, 0.2, \ldots, 1\}$, where $\lambda = 0.0$ means training the network without supervision by the structure-preserving loss, and the classification results are illustrated in Fig. 3. Fig. 3(a) shows that the OA of the Salinas dataset greatly improves as λ increases at the beginning. This is because the structure-preserving loss can make the deep spectral–spatial features maintain the manifold structure of original HSI, which, indeed, enhances the representative ability and the discrimination. The OA of SPSSN on the Salinas dataset begins to fluctuate after the initial increase. For the other three datasets, the OA first increases and then decreases with fluctuation but is always far superior to the counterpart supervised only by cross-entropy loss.

Next, we explore the impact of k on the OA of SPSSN. The optimal value of k is selected within the given set $\{5, 10, 15, 20\}$. The classification results with different values of k are shown in Table II. As can be seen, the SPSSN achieves the best performance when $k = 10$ for Salinas, $k = 10$ for

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:30:16 UTC from IEEE Xplore. Restrictions apply.

TABLE II
OA (%) OF SPSSN WITH DIFFERENT SIZES OF THE BUFFER SET FOR EACH CLASS

k	5	10	15	20
Salinas	98.59	**98.78**	98.72	98.71
Indian Pines	99.06	**99.20**	99.16	99.16
Houston2013	87.68	**87.84**	87.75	87.48
Pavia University	**99.47**	99.42	99.36	99.36

TABLE III
OA (%) OF SPSSN WITH DIFFERENT NUMBERS OF CONVOLUTIONAL KERNEL

C	25	50	75	100	125	150
Salinas	98.11	98.35	98.71	**98.78**	98.74	98.73
Indian Pines	98.77	99.12	99.17	**99.20**	99.16	99.13
Houston2013	87.16	87.21	**87.84**	87.64	87.73	87.21
Pavia University	99.25	**99.47**	99.38	99.33	99.35	99.36

Indian Pines, $k = 10$ for Houston2013, and $k = 5$ for Pavia University, respectively. Therefore, we set them empirically in the following experiments.

Finally, the influence of parameter C on the SPSSN is analyzed. We find the optimal value of C from the set $\{25, 50, \ldots, 150\}$. Table III shows the OAs of the SPSSN with the different C on four datasets. It can be seen from Table III, that as C increases, the OAs first increase and then fluctuate with relatively decent accuracy. The best results on the Salinas and Indian Pines datasets are obtained when $C = 100$. As for the Pavia University dataset, the optimal number of convolutional kernels is $C = 50$. The optimal number of convolutional kernels is $C = 75$ for the Houston2013 dataset. Moreover, we also discover an interesting phenomenon: the optimal numbers of convolutional kernels on four datasets are about half of the spectral dimensions of the corresponding HSIs.

C. Comparison of Different Methods

The SPSSN is compared with several state-of-the-art deep learning-based methods: the CCNN [20], 2D-CNN [19], DFFN [22], and SSRN [23]. In addition, the SSN, which has the same network architecture but is supervised only by cross-entropy loss, is also used as a benchmark. All methods do not employ the dimension reduction and use the original HSI cubes with the spatial size of 7×7 as network inputs. We set other hyperparameters of compared methods to the same as respective papers. For the Salinas, Indian Pines, and Pavia University datasets, 1%, 10%, and 1% labeled HSI cubes of each class are randomly selected as the training set, respectively. For the Houston2013 dataset, the training samples and the testing samples are fixed.

1) Salinas Dataset: Table IV presents the quantitative experimental results of various classification methods on the Salinas dataset. The SPSSN demonstrates the overall superiority with the best OA, AA, and κ. In addition, the SPSSN obtains the smallest standard deviation of OA and also shows the advantages in classifying the majority of classes (i.e., 13 out of 16 classes). All these demonstrate the effectiveness

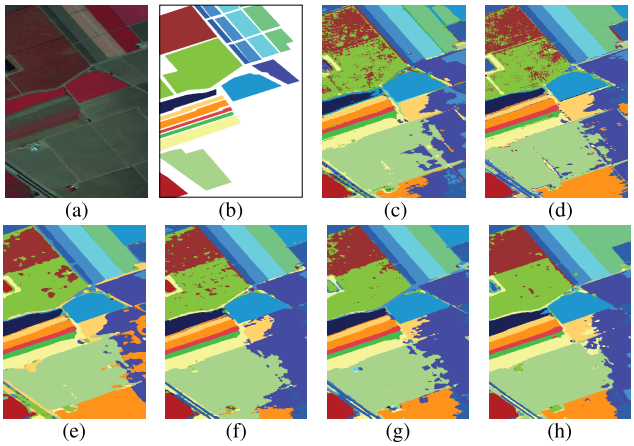


Fig. 4. Visualization of classification maps on Salinas dataset. (a) False color image. (b) Ground truth. (c) CCNN. (d) DFFN. (e) 2D-CNN. (f) SSRN. (g) SSN. (h) SPSSN.

of the proposed SPSSN. It is worth noting that the SSRN, SSN, and SPSSN built SSN architectures to jointly model spectral information and spatial information, which gives very high classification accuracies and shows the potential of this strategy. Moreover, the SPSSN increases the performance of SSN by 0.88%, 0.6%, and 0.97% in OA, AA, and κ, which indicates that the structure-preserving loss, indeed, contributes to the deep features' learning. This is because, during the training procedure, the SPSSN not only makes use of the label information but also takes the structure of the input data into consideration and transfers the structure information to the learned deep features. In this way, the structure of the input data can be further utilized in SPSSN, while, in SSN, the learned deep features are totally uncontrolled and do not retain the original data structure.

Fig. 4 illustrates the classification maps of various methods on the whole Salinas scene. As shown in Fig. 4(b), the Salinas dataset comprises many homogeneous regions. Due to lots of misclassifications, the CCNN, DFFN, and 2D-CNN contain much noise in their classification results. Through explicitly exploring spectral information and spatial information, the SSRN and SSN can effectively remove most noisy labels from classification maps. It is obvious that the SPSSN has the best visual performance, which can preserve the smoothness in most homogeneous regions well.

The above analysis indicates that the SPSSN method shows a great superiority and emphasizes the significance of the structure-preserving loss.

2) Indian Pines Dataset: Table V reports the classification accuracies of different methods for the Indian Pines dataset. The SPSSN outperforms other methods in terms of OA and κ and also shows the advantages in classifying the majority of classes (i.e., ten out of sixteen classes). These demonstrate the effectiveness of the proposed method. Note that the SPSSN can significantly improve the accuracies of the classes with sufficient training samples but shows relatively poor classification accuracies about the classes with fewer training data. As a result, the SPSSN exhibits distinct advantages in OA and κ, but it is slightly inferior to SSN for AA. One possible reason is that the classes with extremely fewer training samples cannot

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:30:16 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
QUANTITATIVE CLASSIFICATION ACCURACIES ON THE SALINAS DATASET

Class	train	test	CCNN	DFFN	2D-CNN	SSRN	SSN	SPSSN
Weeds-1	21	1988	98.75	89.27	**99.80**	99.57	99.10	99.61
Weeds-2	38	3688	99.43	99.35	99.79	**100.00**	99.98	**100.00**
Fallow	20	1956	98.04	99.61	98.92	**100.00**	99.78	**100.00**
Fallow-RP	14	1380	98.20	97.57	98.22	**99.46**	98.86	99.06
Fallow-S	27	2651	96.68	99.26	98.90	**99.88**	98.68	99.06
Stubble	40	3919	99.90	99.95	99.96	**100.00**	**100.00**	**100.00**
Celery	36	3543	99.27	99.46	99.77	**99.99**	99.96	**99.99**
Grapes	113	11158	83.81	85.12	91.31	94.23	96.08	**97.17**
Soil	63	6140	99.31	99.35	99.91	**100.00**	99.95	**100.00**
Corn	33	3245	93.33	94.21	95.75	98.51	97.84	**99.06**
Lettuce-4	11	1057	94.59	92.99	96.89	99.18	98.63	**99.55**
Lettuce-5	20	1907	99.22	99.97	99.91	**100.00**	**100.00**	**100.00**
Lettuce-6	10	906	98.74	99.13	98.96	99.30	99.16	**99.69**
Lettuce-7	11	1059	97.54	96.49	98.13	98.29	98.67	**98.99**
Vinyard-U	73	7195	70.81	77.95	89.51	90.94	93.27	**96.67**
Vinyard-VT	19	1788	95.18	97.43	97.10	99.60	99.07	**99.75**
OA(%)	-	-	91.44±0.54	92.61±1.38	96.12±0.53	97.38±0.29	97.90±0.29	**98.78±0.25**
AA(%)	-	-	95.17±0.43	95.44±1.88	97.68±0.52	98.68±0.16	98.69±0.32	**99.29±0.18**
$\kappa \times 100$	-	-	90.46±0.60	91.77±1.54	95.68±0.59	97.08±0.32	97.67±0.32	**98.64±0.27**

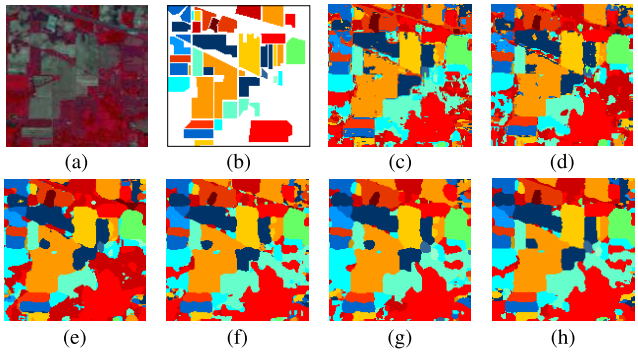


Fig. 5. Visualization of classification maps on Indian Pines dataset. (a) False color image. (b) Ground truth. (c) CCNN. (d) DFFN. (e) 2D-CNN. (f) SSRN. (g) SSN. (h) SPSSN.

provide sufficient structure information for the SPSSN, which decreases the classification accuracies of these classes.

Fig. 5 shows the visual classification maps of all methods. It can be observed that the CCNN produces a noisy classification map. Although the DFFN, 2D-CNN, and SSRN obtain better visual maps, they cannot preserve the semantically meaningful spatial structure and also lose some useful details. In contrast, the SPSSN shows obvious superiority in keeping the edge and local details of the original HSI. Compared with the SSN, the SPSSN can maintain local spatial structure well. This is because the SPSSN can make the learned deep features capture the structured information contained in the HSI, which is helpful for precise classification.

3) Houston2013 Dataset: Table VI displays the quantitative accuracies of different deep methods on the Houston2013 dataset. The CCNN, DFFN, and 2D-CNN yield relatively poor classification accuracies because of many challenging classes and also more complex spatial variations. In contrast, the SSN-based deep methods (i.e., SSRN, SSN, and SPSSN) improve the classification performance dramatically, which verifies the effectiveness of designing spectral and spatial modules to explicitly utilize spectral information and spatial information. When comparing SSRN and SSN, it illustrates that the network architecture of the proposed method is more effective than that of SSRN in terms of modeling the spectral properties and the spatial structural information. Remarkably, the proposed SPSSN outperforms the other methods overall in terms of OA, AA, and κ and also produces satisfactory accuracies in the challenging classes, such as "Commercial," "Highway," and "Railway." Furthermore, in contrast to SSN, the SPSSN improves the OA, AA, and κ by 1.54%, 1.3%, and 1.67%, respectively. This means that the structure-preserving loss, indeed, enhances the discrimination ability of features.

The visual comparison of the classification maps on the Houston2013 dataset is visualized in Fig. 6. The training and testing samples are fixed for the Houston2013 dataset, which is also shown in Fig. 6. It can be observed that the results of CCNN, DFFN, and 2D-CNN are a little noisy, while the SSRN and SSN can remove much noise and improve the visual performance. This is consistent with the quantitative classification results and demonstrates the potential of the strategy of designing the SSN architectures again. The SPSSN shows a superior visual result, which preserves the visual smoothness in the homogeneous regions and also maintains the sharpness of HSI edges.

4) Pavia University Dataset: The experiment is finally carried out on the Pavia University dataset. This scene is challenging due to higher spatial resolution and also because of complex spatial distribution. Table VII summarizes the individual class accuracies, OA, AA, and κ of various methods. It can be seen that our SPSSN obtains the best OA, AA, and κ with the smallest standard deviations for this dataset. This validates the effectiveness and the overall superiority of the proposed SPSSN. Note that other SSN-based deep methods also achieve significantly better classification results, which illustrates the effectiveness of spectral and spatial modules once again.

The visual results on the Pavia University scene are illustrated in Fig. 7, which also highlights the superiority of the SPSSN. The CCNN and DFFN tend to generate classification maps with obvious noise, while the classification result

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:30:16 UTC from IEEE Xplore. Restrictions apply.

TABLE V
QUANTITATIVE CLASSIFICATION ACCURACIES ON THE INDIAN PINES DATASET

Class	train	test	CCNN	DFFN	2D-CNN	SSRN	SSN	SPSSN
Alfalfa	5	41	79.51	73.17	90.98	94.63	**97.56**	96.83
Corn-N	143	1285	93.41	94.80	96.42	97.42	98.65	**99.30**
Corn-M	83	747	94.98	95.84	98.19	97.91	98.53	**99.54**
Corn	24	213	87.28	89.44	96.06	98.50	**98.83**	98.26
Grass-P	49	434	94.77	94.33	95.16	96.01	96.91	**97.67**
Grass-T	73	657	97.98	98.81	98.93	99.39	99.04	**99.76**
Grass-PM	3	25	81.60	78.40	**98.40**	92.40	96.80	91.60
Hay-W	48	430	99.30	99.44	99.98	99.70	**100.00**	**100.00**
Oats	2	18	62.22	83.89	**89.44**	85.56	85.00	79.44
Soybean-N	98	874	90.02	92.51	95.19	97.05	97.81	**98.75**
Soybean-M	246	2209	95.86	96.91	98.37	98.29	99.00	**99.49**
Soybean-C	60	533	86.36	89.85	97.50	96.72	97.65	**98.82**
Wheat	21	184	99.40	99.35	**100.00**	99.78	98.42	99.84
Woods	127	1138	97.58	97.38	98.80	99.20	99.86	**99.94**
Buildings	39	347	86.80	90.58	94.12	97.00	99.08	**99.71**
Stone	10	83	94.82	97.11	**99.64**	97.47	92.05	90.48
OA(%)	-	-	94.16±0.93	95.35±1.24	97.53±0.42	98.00±0.52	98.67±0.40	**99.20±0.32**
AA(%)	-	-	90.12±1.96	91.99±2.66	96.70±0.94	96.69±1.18	**97.20±1.48**	96.84±1.62
$\kappa \times 100$	-	-	93.34±1.07	94.70±1.41	97.18±0.48	97.72±0.59	98.48±0.45	**99.09±0.37**

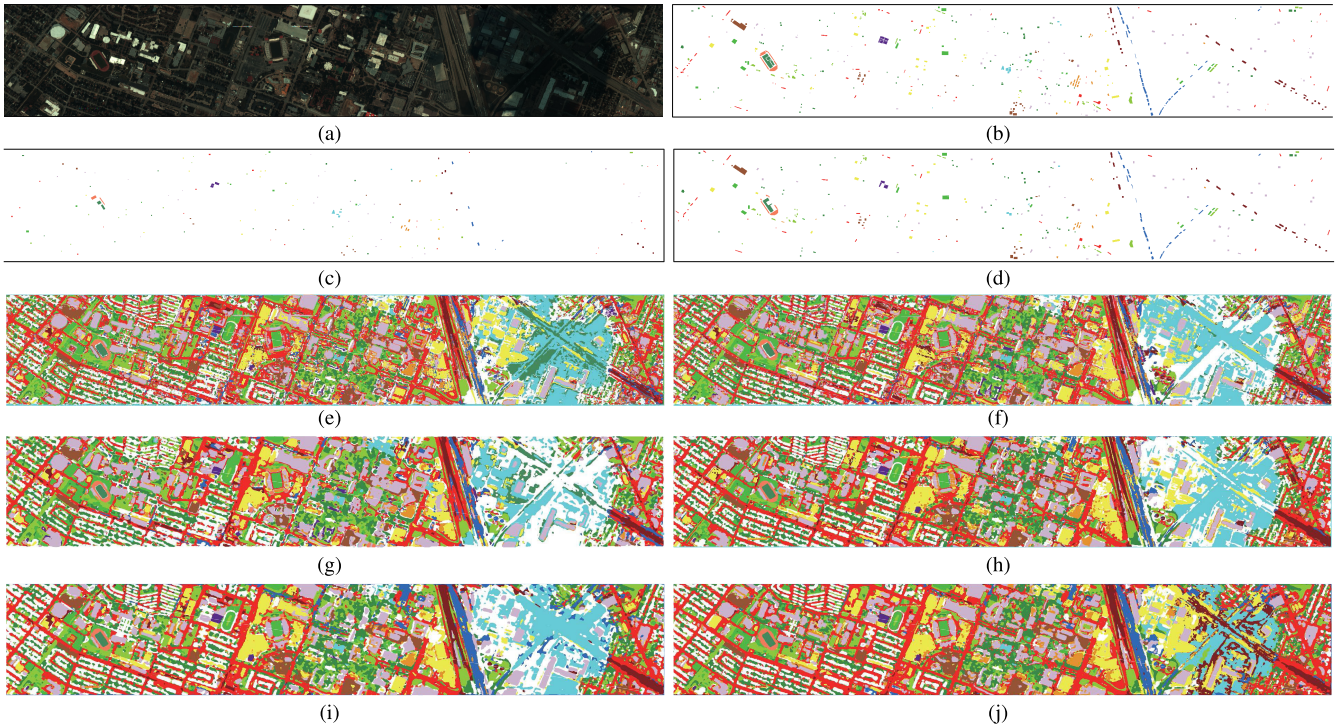


Fig. 6. Visualization of classification maps on Houston2013 dataset. (a) False color image. (b) Ground truth. (c) Training samples. (d) Testing samples. (e) CCNN. (f) DFFN. (g) 2D-CNN. (h) SSRN. (i) SSN. (j) SPSSN.

of 2D-CNN is oversmoothing. In contrast, the other three SSN-based deep methods can obtain refined results. It can be found that our SPSSN achieves better visual quality than other compared methods, in which the homogeneous regions seem much smoother, and the image boundaries are distinct.

In summary, the SPSSN obtains improved performance over these baselines on four datasets. The OAs of SPSSN are better than the best-cited method by 1.4%, 1.2%, 2.25%, and 1.03% for the Salinas, Indian Pines, Houston2013, and Pavia University datasets, respectively. This is achieved mainly with the help of the proposed structure-preserving loss for the representative ability enhancement. The proposed SSN also helps to extract discriminative feature representation for HSI classification.

D. Running Time

Table VIII displays the training time, the testing time, and the number of parameters for all methods. The CCNN, DFFN, and 2D-CNN spend less time during the training stage, while the SSN-based methods (i.e., SSRN, SSN, and SPSSN)

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:30:16 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
QUANTITATIVE CLASSIFICATION ACCURACIES ON THE HOUSTON2013 DATASET

Class	train	test	CCNN	DFFN	2D-CNN	SSRN	SSN	SPSSN
Healthy grass	198	1053	82.41	82.02	81.78	82.82	82.71	**84.46**
Stressed grass	190	1064	83.82	84.44	83.83	84.83	85.14	**88.97**
Synthetic grass	192	505	83.03	86.38	96.10	**99.94**	99.90	99.07
Trees	188	1056	89.00	91.99	91.55	95.21	94.00	**95.37**
Soil	186	1056	98.86	99.58	99.79	99.96	**100.00**	**100.00**
Water	182	143	94.34	95.10	95.59	96.36	97.20	**97.97**
Residential	196	1072	85.42	83.48	**90.98**	87.19	86.84	84.69
Commercial	191	1053	61.06	73.26	60.76	65.67	75.88	**80.67**
Road	193	1059	**87.66**	77.21	77.13	82.63	86.99	86.68
Highway	191	1036	52.29	59.02	55.92	56.90	57.75	**59.06**
Railway	181	1054	63.97	67.13	64.62	83.95	82.04	**85.44**
Parking Lot 1	192	1041	79.49	85.32	85.84	**97.43**	93.16	96.77
Parking Lot 2	184	285	87.96	88.39	80.77	88.46	89.96	**91.16**
Tennis Court	181	247	85.26	91.38	94.74	**100.00**	**100.00**	**100.00**
Running Track	187	473	90.15	92.73	99.96	**100.00**	99.22	99.96
OA(%)	-	-	79.66±0.57	81.70±0.53	81.32±0.33	85.59±0.51	86.30±0.42	**87.84±0.61**
AA(%)	-	-	81.65±0.69	83.83±0.71	83.96±0.38	88.09±0.46	88.72±0.37	**90.02±0.45**
$\kappa \times 100$	-	-	77.97±0.62	80.25±0.56	79.79±0.36	84.44±0.55	85.17±0.46	**86.84±0.65**

TABLE VII
QUANTITATIVE CLASSIFICATION ACCURACIES ON THE PAVIA UNIVERSITY DATASET

Class	train	test	CCNN	DFFN	2D-CNN	SSRN	SSN	SPSSN
Asphalt	67	6564	91.55	92.65	97.01	98.56	98.90	**99.75**
Meadows	187	18462	98.75	98.64	99.50	99.46	99.97	**100.00**
Gravel	21	2078	67.05	79.30	89.03	88.35	92.93	**96.02**
Trees	31	3033	92.45	92.77	93.86	**97.85**	96.44	**97.85**
Sheets	14	1331	98.44	98.68	99.53	**100.00**	99.95	**100.00**
Soil	51	4978	84.06	90.77	96.22	98.11	99.63	**99.86**
Bitumen	14	1316	68.59	77.63	91.93	98.54	**99.00**	98.68
Bricks	37	3645	80.72	86.21	95.02	98.77	97.99	**99.15**
Shadows	10	937	87.20	96.44	99.16	**99.94**	98.95	99.42
OA(%)	-	-	91.15±0.85	93.65±1.40	97.18±0.62	98.44±0.60	98.94±0.28	**99.47±0.22**
AA(%)	-	-	85.42±1.79	90.34±2.01	95.69±1.00	97.73±0.55	98.20±0.67	**98.97±0.45**
$\kappa \times 100$	-	-	88.14±1.17	91.54±1.86	96.26±0.82	97.93±0.80	98.60±0.37	**99.29±0.29**

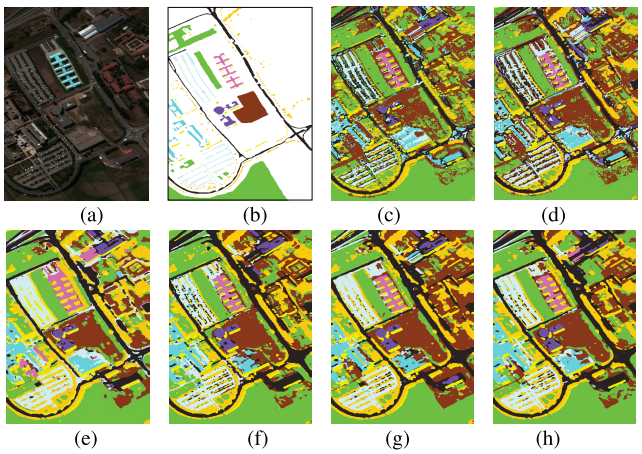


Fig. 7. Visualization of classification maps on Pavia University dataset. (a) False color image. (b) Ground truth. (c) CCNN. (d) DFFN. (e) 2D-CNN. (f) SSRN. (g) SSN. (h) SPSSN.

take six to ten times longer for training. That is mainly because these SSN-based methods utilize a large number of 3-D convolutional kernels, and the computational efficiency of 3-D convolutions is low, which results in more training time and testing time. Compared with the SSN, the SPSSN has a higher computational cost at the training stage, and the main reason is that the SPSSN spends additional time to calculate the structure-preserving loss during each training iteration. For the testing procedure, the SPSSN is as efficient as the SSN. It can be concluded from Table VIII that the training time of our SPSSN is a potential drawback, and this could be refined by the graphical processing unit (GPU) with much higher performance.

E. Effect of Different Spatial Sizes and Training Percentages

1) Spatial Size: The effect of different spatial sizes on the performance of the proposed SPSSN is explored. The spatial size of the HSI cube, i.e., s, varies from 3 to 11 with a step of 2. Table IX illustrates the OAs of the SPSSN under different spatial sizes on the Salinas, Indian Pines, Houston2013, and Pavia University datasets.

It can be seen that the OA of the proposed SPSSN improves as the spatial size increases at the beginning. This is because the HSI cube with a large spatial size can contain more spatial information. However, when the spatial size is more than 7, the OAs of the SPSSN slightly improve or even decrease on the Indian Pines and Pavia University datasets. The reason is

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:30:16 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
TRAINING TIME, TESTING TIME, AND THE NUMBER OF PARAMETERS ON FOUR DATASETS

Methods	Salinas			Indian Pines			Houston2013			Pavia University		
	Train(m)	Test(s)	Parameters	Train(m)	Test(s)	Parameters	Train(m)	Test(s)	Parameters	Train(m)	Test(s)	Parameters
CCNN	23.52	13.03	2,334,608	16.06	2.81	2,292,112	13.16	4.09	1,697,039	12.35	5.05	1,260,681
DFFN	26.77	14.16	446,272	18.35	3.17	445,696	17.98	6.19	437,567	18.74	7.47	431,273
2D-CNN	16.43	12.67	35,536	8.99	2.36	35,024	8.05	2.16	27,791	8.01	4.89	22,153
SSRN	117.49	37.50	369,576	92.27	6.45	363,432	71.70	7.73	277,391	57.13	13.63	215,801
SSN	172.59	56.46	812,916	137.93	9.76	802,916	78.59	8.40	373,365	41.46	11.69	141,109
SPSSN	189.33	57.18	812,916	153.86	9.76	802,916	93.51	8.47	373,365	52.22	11.81	141,109

TABLE IX
OA (%) OF SPSSN WITH DIFFERENT SPATIAL SIZES

$s \times s$	3×3	5×5	7×7	9×9	11×11	13×13
Salinas	93.43	96.44	98.78	99.40	99.69	**99.81**
Indian Pines	95.67	99.18	**99.20**	98.94	98.93	98.95
Houston2013	85.59	87.64	87.84	**88.00**	87.74	87.27
Pavia University	97.01	98.88	99.47	99.58	**99.63**	99.57

TABLE X
OA (%) OF ALL METHODS WITH DIFFERENT TRAINING PERCENTAGES

Dataset	$p(\%)$	CCNN	DFFN	2D-CNN	SSRN	SSN	SPSSN
Salinas	0.5	89.20	91.23	95.79	95.83	97.19	**97.43**
	1	91.44	92.61	96.12	97.38	97.90	**98.78**
	2	94.15	95.67	96.76	98.28	99.13	**99.27**
	5	96.30	98.55	97.39	99.61	99.80	**99.92**
Indian Pines	5	88.27	89.38	95.79	97.02	97.58	**97.93**
	10	94.16	95.35	97.53	98.00	98.67	**99.20**
	15	96.38	97.83	97.90	99.41	99.44	**99.56**
	20	98.61	98.85	98.43	99.62	99.77	**99.89**
Pavia University	0.5	86.75	89.94	94.95	98.25	98.78	**99.31**
	1	91.15	93.65	97.18	98.44	98.94	**99.47**
	2	94.58	97.46	98.68	99.23	99.55	**99.76**
	5	97.71	99.00	99.18	99.76	99.86	**99.89**

that, if the spatial size is very large, the HSI cube may contain many hyperspectral pixels whose class labels are different from that of its center pixel, which decreases the classification performance. Moreover, the classification map produced by the SPSSN with the large spatial size is over smooth, and too large spatial size also results in a significant increase in computational burden. Therefore, a moderate spatial size should be chosen, in practice, to give satisfactory accuracies and acceptable computing burden.

2) Training Percentage: Then, the experiment is conducted to discuss the influence of different training percentages on the classification performance of all methods. We randomly select $p\%$ ($p = 0.5, 1.0, 2.0, 5.0$) labeled HSI cubes in each class as the training data for the Salinas and Pavia University datasets. As for the Indian Pines dataset, $p\%$ ($p = 5, 10, 15, 20$) labeled HSI cubes per class are randomly considered as the training samples. The corresponding results are reported in Table X. It can be seen that the progressively increased number of training samples can lead to significant improvement in the OA of all methods. The proposed SPSSN consistently performs the best among all methods under all these situations. In particular, the SPSSN can present excellent performance when the number of training samples is very limited.

IV. CONCLUSION

This article presents an SPSSN to extract discriminative deep structure-preserving spectral–spatial features for HSI classification. First, the SPSSN utilizes successive spectral module and spatial module to make full use of the spectral–spatial information in HSI cubes. Second, the SPSSN imposes the structure-preserving constraint on the feature learning procedure, which can transfer the local structure of input data into the learned deep spectral–spatial features and enhance the representative ability. Extensive experiments and comparisons on widely used HSI datasets demonstrate that the proposed SPSSN outperforms other state-of-the-art methods in terms of both visual quality and quantitative metrics. In future work, we will study how to combine the structure-preserving constraint and deep learning in the unsupervised framework.

REFERENCES

[1] J. Peng, W. Sun, and Q. Du, "Self-paced joint sparse representation for the classification of hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 1183–1194, Feb. 2019.

[2] H. Ye, H. Li, B. Yang, F. Cao, and Y. Tang, "A novel rank approximation method for mixture noise removal of hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 7, pp. 4457–4469, Jul. 2019.

[3] F. Luo, B. Du, L. Zhang, L. Zhang, and D. Tao, "Feature learning using spatial–spectral hypergraph discriminant analysis for hyperspectral image," *IEEE Trans. Cybern.*, vol. 49, no. 7, pp. 2406–2419, Jul. 2019.

[4] J. Peng, Y. Zhou, W. Sun, Q. Du, and L. Xia, "Self-paced nonnegative matrix factorization for hyperspectral unmixing," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 2, pp. 1501–1515, Feb. 2021.

[5] X. Kang, X. Xiang, S. Li, and J. A. Benediktsson, "PCA-based edge-preserving features for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 12, pp. 7140–7151, Dec. 2017.

[6] P. Ghamisi, M. D. Mura, and J. A. Benediktsson, "A survey on spectral–spatial classification techniques based on attribute profiles," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 5, pp. 2335–2353, May 2015.

[7] J. Peng, L. Li, and Y. Y. Tang, "Maximum likelihood estimation-based joint sparse representation for the classification of hyperspectral remote sensing images," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 6, pp. 1790–1802, Jun. 2019.

[8] Z. Ye, J. Chen, H. Li, Y. Wei, G. Xiao, and J. A. Benediktsson, "Supervised functional data discriminant analysis for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 2, pp. 841–851, Feb. 2020.

[9] W. Sun, J. Peng, G. Yang, and Q. Du, "Fast and latent low-rank subspace clustering for hyperspectral band selection," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 6, pp. 3906–3915, Jun. 2020.

[10] J. M. Haut, M. E. Paoletti, J. Plaza, A. Plaza, and J. Li, "Visual attention-driven hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 10, pp. 8065–8080, Oct. 2019.

[11] X. Cao, F. Zhou, L. Xu, D. Meng, Z. Xu, and J. Paisley, "Hyperspectral image classification with Markov random fields and a convolutional neural network," *IEEE Trans. Image Process.*, vol. 27, no. 5, pp. 2354–2367, May 2018.

[12] G. Cheng, Z. Li, J. Han, X. Yao, and L. Guo, "Exploring hierarchical convolutional features for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 11, pp. 6712–6722, Nov. 2018.

[13] W. Guo, H. Ye, and F. Cao, "Features-grouped network with spectral-spatial connected attentions for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, early access, Jan. 20, 2021, doi: 10.1109/TGRS.2021.3051056.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:30:16 UTC from IEEE Xplore. Restrictions apply.

[14] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.
[15] W. Hu, Y. Huang, L. Wei, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, Jan. 2015, Art. no. 258619.
[16] L. Mou, P. Ghamisi, and X. X. Zhu, "Deep recurrent neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3639–3655, Jul. 2017.
[17] W. Li, G. Wu, F. Zhang, and Q. Du, "Hyperspectral image classification using deep pixel-pair features," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 2, pp. 844–853, Feb. 2017.
[18] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.
[19] S. Yu, S. Jia, and C. Xu, "Convolutional neural networks for hyperspectral image classification," *Neurocomputing*, vol. 219, pp. 88–98, Jan. 2017.
[20] H. Lee and H. Kwon, "Going deeper with contextual CNN for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 26, no. 10, pp. 4843–4855, Oct. 2017.
[21] J. Zhu, L. Fang, and P. Ghamisi, "Deformable convolutional neural networks for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 8, pp. 1254–1258, Aug. 2018.
[22] W. Song, S. Li, L. Fang, and T. Lu, "Hyperspectral image classification with deep feature fusion network," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 6, pp. 3173–3184, Jun. 2018.
[23] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.
[24] H. Sun, X. Zheng, X. Lu, and S. Wu, "Spectral–spatial attention network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3232–3245, May 2020.
[25] F. Cao and W. Guo, "Cascaded dual-scale crossover network for hyperspectral image classification," *Knowl.-Based Syst.*, vol. 189, Feb. 2020, Art. no. 105122.
[26] B. Liu, X. Yu, P. Zhang, A. Yu, Q. Fu, and X. Wei, "Supervised deep feature extraction for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 4, pp. 1909–1921, Apr. 2018.
[27] A. J. X. Guo and F. Zhu, "Spectral–spatial feature extraction and classification by ANN supervised with center loss in hyperspectral imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 3, pp. 1755–1767, Mar. 2019.
[28] B. Yang, H. Li, and Z. Guo, "Learning a deep similarity network for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 1482–1496, 2021.
[29] B. Deng, S. Jia, and D. Shi, "Deep metric learning-based feature embedding for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 2, pp. 1422–1435, Feb. 2020.
[30] X. Lu, H. Wu, Y. Yuan, P. Yan, and X. Li, "Manifold regularized sparse NMF for hyperspectral unmixing," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 5, pp. 2815–2826, May 2013.
[31] C. Li *et al.*, "Deep manifold structure transfer for action recognition," *IEEE Trans. Image Process.*, vol. 28, no. 9, pp. 4646–4658, Sep. 2019.
[32] B. Zhang, W. Yang, Z. Wang, L. Zhuo, J. Han, and X. Zhen, "The structure transfer machine theory and applications," *IEEE Trans. Image Process.*, vol. 29, pp. 2889–2902, 2020.
[33] Y. Zhou and Y. Wei, "Learning hierarchical spectral–spatial features for hyperspectral image classification," *IEEE Trans. Cybern.*, vol. 46, no. 7, pp. 1667–1678, Jul. 2016.
[34] Y. Zhou, J. Peng, and C. L. P. Chen, "Dimension reduction using spatial and spectral regularized local discriminant embedding for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 2, pp. 1082–1095, Feb. 2015.
[35] G. Shi, H. Huang, and L. Wang, "Unsupervised dimensionality reduction for hyperspectral imagery via local geometric structure feature learning," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 8, pp. 1425–1429, Aug. 2020.
[36] X. Zheng, Y. Yuan, and X. Lu, "Dimensionality reduction by spatial–spectral preservation in selected bands," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 9, pp. 5185–5197, Sep. 2017.
[37] Q. Wang, X. He, and X. Li, "Locality and structure regularized low rank representation for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 911–923, Feb. 2019.
[38] M. Imani and H. Ghassemian, "Morphology-based structure-preserving projection for spectral–spatial feature extraction and classification of hyperspectral data," *IET Image Process.*, vol. 13, no. 2, pp. 270–279, Feb. 2019.
[39] H. Huang, G. Shi, H. He, Y. Duan, and F. Luo, "Dimensionality reduction of hyperspectral imagery based on spatial–spectral manifold learning," *IEEE Trans. Cybern.*, vol. 50, no. 6, pp. 2604–2616, Jun. 2020.
[40] Y. Yuan, J. Fang, X. Lu, and Y. Feng, "Spatial structure preserving feature pyramid network for semantic image segmentation," *ACM Trans. Multimedia Comput., Commun., Appl.*, vol. 15, no. 3, p. 73, 2019.
[41] Z. Li, H. Huang, Y. Li, and Y. Pan, "M^{3}DNet: A manifold-based discriminant feature learning network for hyperspectral imagery," *Expert Syst. Appl.*, vol. 144, Apr. 2020, Art. no. 113089.
[42] Z. Li, H. Huang, Y. Duan, and G. Shi, "DLPNet: A deep manifold network for feature extraction of hyperspectral imagery," *Neural Netw.*, vol. 129, pp. 7–18, Sep. 2020.
[43] M. Belkin and P. Niyogi, "Laplacian eigenmaps for dimensionality reduction and data representation," *Neural Comput.*, vol. 15, no. 6, pp. 1373–1396, Jun. 2003.
[44] R. Sinha, "Whitney embedding theorem," in *Smooth Manifolds*. New Delhi, India: Springer, 2014, pp. 477–483.
[45] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent.*, May 2015, pp. 1–15.



Bing Yang received the B.Sc. degree in information and computing science from Henan University, Kaifeng, China, in 2015. He is pursuing the Ph.D. degree with the School of Mathematics and Statistics, Huazhong University of Science and Technology, Wuhan, China.

His research interests include pattern recognition and machine learning.



Hong Li (Member, IEEE) received the M.Sc. degree in mathematics and the Ph.D. degree in pattern recognition and intelligence control from the Huazhong University of Science and Technology, Wuhan, China, in 1986 and 1999, respectively.

She is a Professor with the School of Mathematics and Statistics, Huazhong University of Science and Technology. Her research interests include approximation theory, wavelet analysis, learning theory, neural networks, signal processing, and pattern recognition.



Ziyang Guo received the B.Sc. and M.Sc. degrees in applied mathematics from the Huazhong University of Science and Technology, Wuhan, China, in 2011 and 2014, respectively, where he is pursuing the Ph.D. degree with the School of Mathematics and Statistics.

His research interests include pattern recognition and machine learning.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:30:16 UTC from IEEE Xplore. Restrictions apply.